Q1:
Use probuilder to make prefab for T and U shape, manually added vertices at each corner and normal info by creating empty at that location and encode degree in y rotation

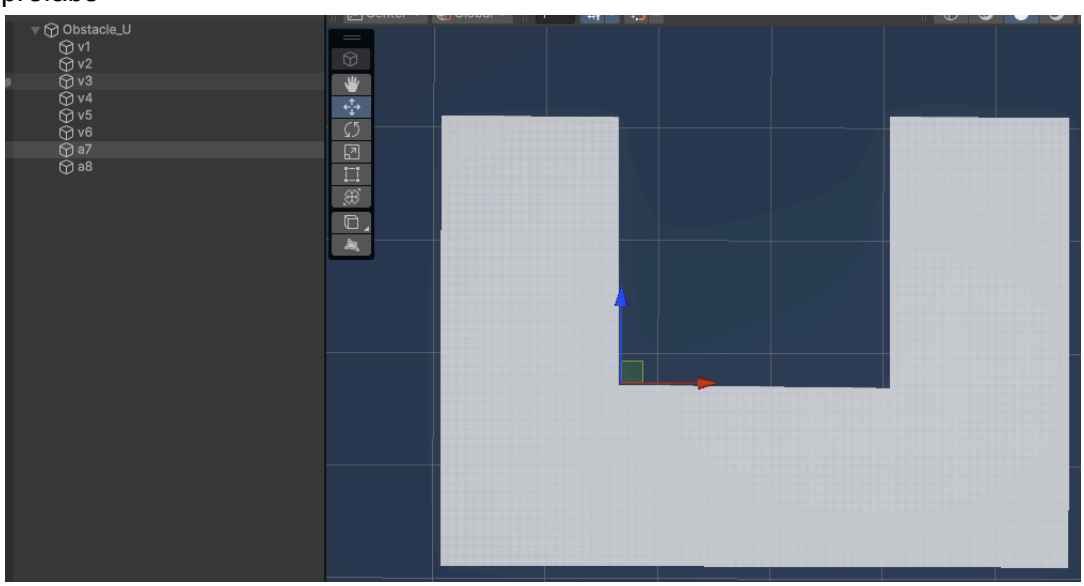
Generate by using margin between each prefab and margin between boundaries, find largest radii of each prefab shape and sum with agent radii to get the margin value

Separate the area in to 6 subarea and give them random value and color for low and high and draw the value

Q2:
Agent generated as a sphere, at start location.
Draw goal location blue

Q3:
Depend on agent radii we do Minkowski sum for each prefab

- Push the vertices of each prefab by direction normal and magnitude radii
- Connect the edges to form new boundaries
- V: Add the reflex vertices by marking them vi and not ai for non reflex vertices for the prefabs

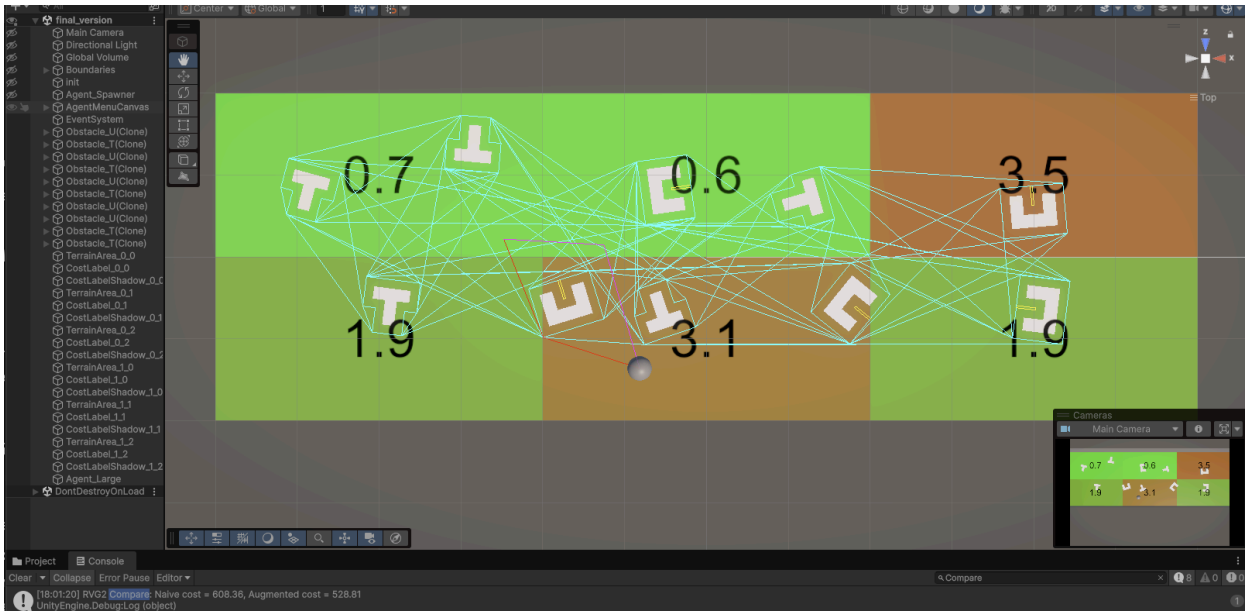


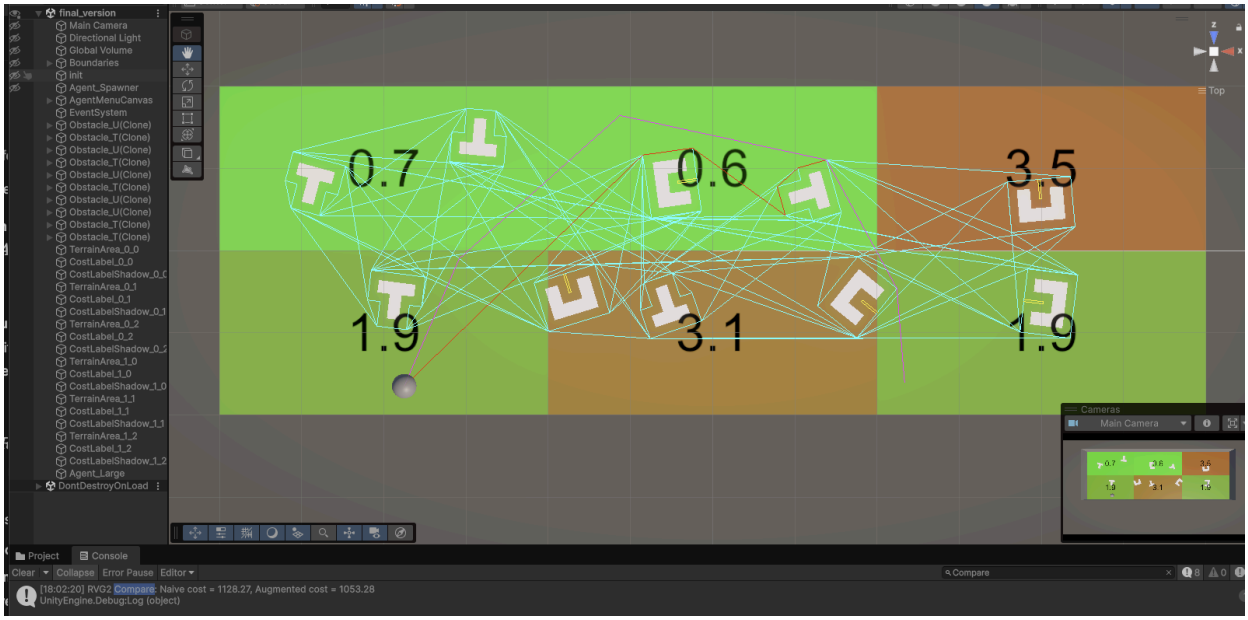
- E: Add bitangents by implement IsVisible function that use orientation(signed area) to check if two vertices form an edge and it will not collide with every other boundaries, and another function to extend the edge on both ends by agent radii and see if they collide with original obstacle boundaries, if they don't they are bitangent and we add them in the graph edges
- We compute the edge weight using the terrain cost.
- Now we have the graph (V,E) based on agent radii and we just run disktra on it.

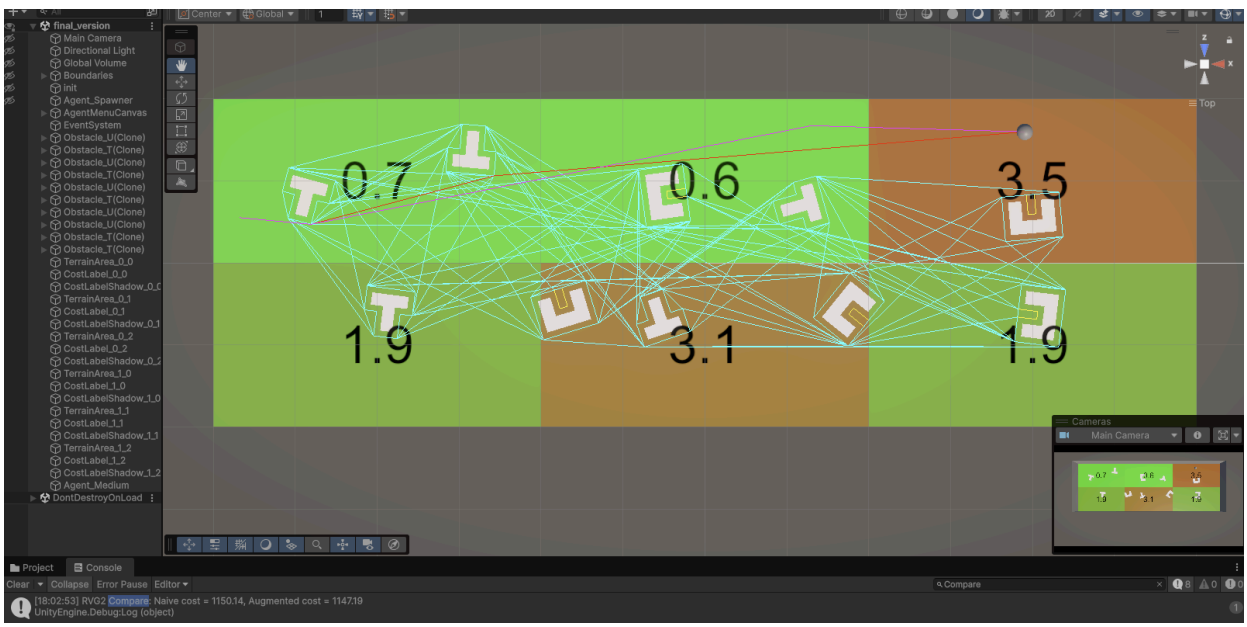
Q4:

Take the RVG graph that we just built, add random vertices and connect them with the RVG graph based on visibility and compute the edge weight.
Now we have the augmentedRVG graph with more vertices. We again just do Dikstra on it. This will return a heuristic solution since we are not adding all points.

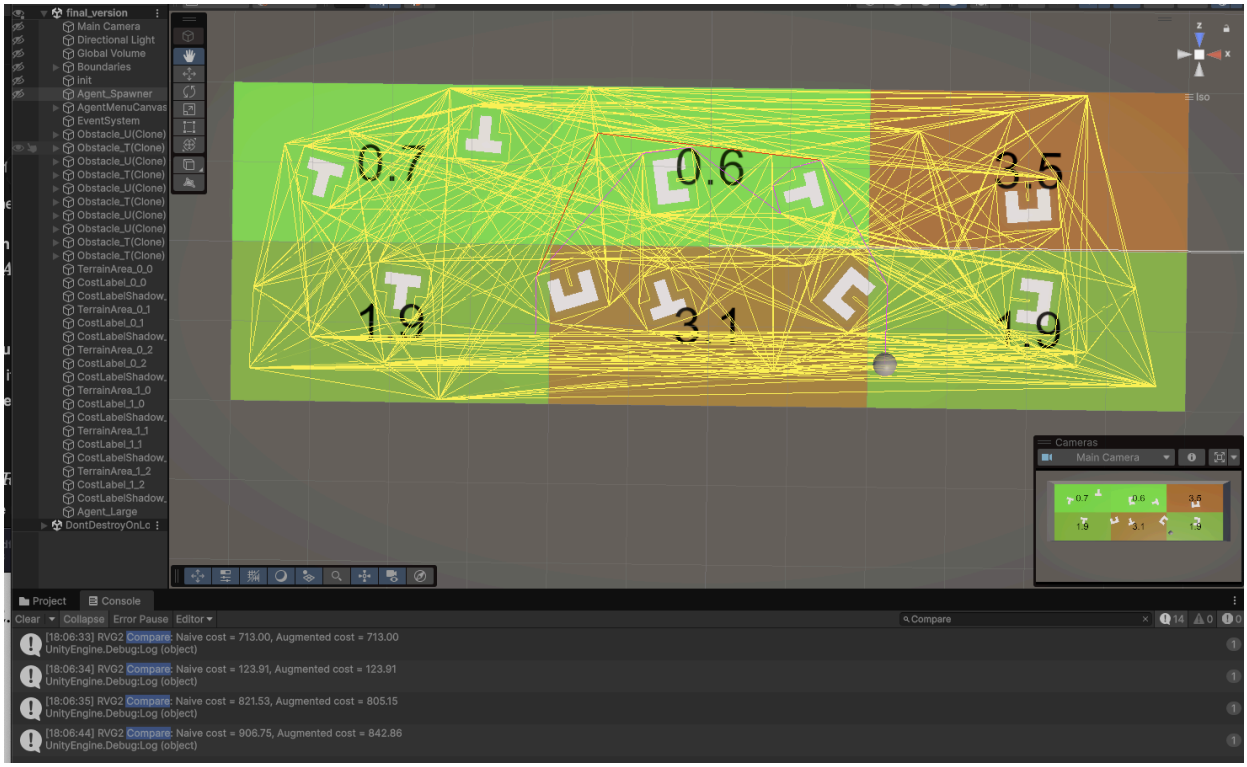
I have used gizmo to show the aug path in pink and naive path in red, when testing you can see it easily and a debug log that shows cost for each. Here are 3 examples.







You can change the mode in the agent_spawner inspector, below is an example of the augmented graph



All features implemented, no limitations